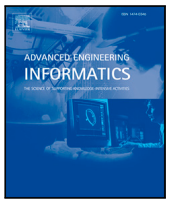
Advanced Engineering Informatics 67 (2025) 103527



Contents lists available at ScienceDirect

Advanced Engineering Informatics

journal homepage: www.elsevier.com/locate/aei





MASC: Large language model-based multi-agent scheduling chain for flexible job shop scheduling problem ☆

Zelong Wang [a,1], Chenhui Wan [a,b,1], Jie Liu [b,c], Xi Zhang [a], Haifeng Wang [a,d], Youmin Hu [a,b], Zhongxu Hu [a,b,*]

[a] School of Mechanical Science and Engineering, Huazhong University of Science and Technology, Wuhan, 430074, China
[b] HUST-Wuxi Research Institute, Huazhong University of Science and Technology, Wuxi, 214100, China
[c] School of Civil and Hydraulic Engineering, Huazhong University of Science and Technology, Wuhan, 430074, China
[d] Wuxi Little Swan Electric Appliance Co., Ltd, Wuxi, 214029, China

ARTICLE INFO

Keywords:
Intelligent manufacturing
Flexible job shop scheduling
Large language model
Multi-agent systems

ABSTRACT

The flexible job shop scheduling problem (FJSP) presents significant challenges in intelligent manufacturing due to the need for efficient resource allocation and dynamic rescheduling. Traditional solutions often require manual intervention and lack real-time adaptability. This paper introduces the Multi-Agent Scheduling Chain (MASC) framework, which leverages large language models (LLMs) for enhanced decision-making and automated equipment control. MASC integrates four agents to manage dynamic scheduling and real-time rescheduling effectively. SchedAgent (Scheduling Agent) uses an improved ReAct method, combining FJSP-specific scheduling indicators and fine-tuned decision algorithms to optimize results. The DialBag (Dialogue Bagging) method is also introduced, constructing a specialized dataset to prevent knowledge loss and enhance decision-making. This method allows the agent to retain knowledge across diverse scheduling contexts while improving performance in specific tasks. MASC was validated through simulations and real-world robotic experiments, handling both machine malfunctions and urgent job additions. These tests demonstrated MASC's robust performance, with significant improvements in scheduling efficiency and rescheduling accuracy. Quantitative results showed that SchedAgent consistently achieved high ranking percentages, with an average ranking percentage of 84% at a 10-second solving time and 90% at a 30-second solving time. MASC provides a scalable and adaptable solution for intelligent manufacturing, demonstrating the potential of LLMs to automate and optimize production workflows in both static and dynamic environments.

1. Introduction

The Flexible Job Shop Scheduling Problem (FJSP) poses a critical challenge in intelligent manufacturing, involving resource allocation, process optimization, and time management across complex production systems [1]. Traditionally, solving FJSP has relied heavily on human expertise to select algorithms [2], optimize parameters [3], and generate comprehensive scheduling plans [4]. These approaches require specialized knowledge and frequent manual interventions, leading to inefficiencies and poor adaptability in dynamic production environments.

In recent years, advancements in artificial intelligence (AI) have facilitated improvements in job shop scheduling [5,6]. Reinforcement learning (RL)-based algorithms [7] iteratively learn strategies by adjusting network parameters, making decisions based on input features, and generating complete schedules through a series of actions. For instance, RL has been successfully applied in human–robot interaction (HRI) within manufacturing, helping robotic systems adapt to variations in human performance and thereby optimizing process efficiency [8]. Early research [9] utilized scheduling rules as RL's action space. Lei et al. [10] proposed a multi-pointer graph network architecture and proximal policy optimization for job and machine selection in FJSP environments, addressing selection problems with hierarchical decisions. However, RL-based approaches have often struggled to handle the semantic complexities of scheduling tasks [11] and adapt to the diverse and real-time demands of modern production environments [12]. These

☆ This work was supported in part by the National Major Science and Technology Projects of China (2024ZD0608100), National Key Research and Development Program of China (No. 2023YFD2100905), National Natural Science Foundation of China (No. 52205104).

* Corresponding author at: HUST-Wuxi Research Institute, Huazhong University of Science and Technology, Wuxi, 214100, China.
E-mail address: zhongxu_hu@hust.edu.cn (Z. Hu).

[1] Zelong Wang and Chenhui Wan contributed equally to this work.

https://doi.org/10.1016/j.aei.2025.103527
Received 23 October 2024; Received in revised form 30 April 2025; Accepted 28 May 2025
Available online 16 June 2025
1474-0346/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

limitations highlight the need for more robust and flexible methods to tackle FJSP challenges effectively.

Recent breakthroughs in Large Language Models (LLMs) have been recognized as promising solutions to these challenges. LLMs, such as ChatGPT [13], have been extensively applied in natural language processing (NLP), demonstrating strong performance in text generation, comprehension, and contextual reasoning. The integration of digital twins [14] and knowledge graphs [15] with LLMs has been investigated to improve real-time decision-making capabilities in dynamic environments, aligning with the principles of cognitive manufacturing [16]. These integrations have been utilized to facilitate intelligent process planning, adaptive scheduling, and knowledge-driven optimization in industrial settings. Moreover, hybrid digital twin architectures have been explored to further enhance the interaction between physical and virtual environments, enabling more robust simulations and predictive analytics [17]. Additionally, graph-based reasoning techniques have been incorporated into scheduling and decision-making processes to improve the interpretability and adaptability of AI-driven systems in complex manufacturing scenarios [18]. By leveraging these advancements, a more comprehensive and data-driven approach to FJSP has been established, ensuring a stronger connection between intelligent scheduling systems and real-world production environments. These capabilities have extended beyond NLP, with LLMs increasingly being applied in fields such as robotic control [19,20], manufacturing, and automation [21,22]. Techniques like prompt engineering and chain-of-thought (CoT) prompting have enabled LLMs to optimize production tasks based on large datasets of production history and scheduling cases [23]. By leveraging this data, LLMs can analyze complex scheduling problems and propose effective optimization strategies, while simultaneously enhancing the process by identifying and addressing potential bottlenecks [24]. Moreover, these models enable direct communication with engineers and automated equipment, streamlining both human-machine collaboration and automation [25].

Despite these advantages, LLMs can produce erroneous responses in unfamiliar domains due to a lack of domain-specific knowledge during pretraining. This phenomenon, known as "AI hallucinations" [26], has been a focus of research, and methods like few-shot prompt engineering [27], CoT prompting [23], and fine-tuning approaches such as Low-Rank Adapters (LoRA) [28] and Quantized Low-Rank Adapters (QLoRA) [29] have been developed to mitigate these issues. These techniques have improved LLM performance, particularly when combined with multi-agent systems (MAS) designed to integrate LLM-driven decision-making across various environments [30–32]. In addition, using knowledge graphs, which represent tasks, resources, and organizational structures, has proven effective for enhancing process optimization and root cause analysis in complex industrial scenarios [33].

Combining LLMs and MAS presents a promising approach to addressing the complexities of FJSP. While early agent-based systems relied on static rule-based methods, recent AI agents, empowered by LLMs, provide more flexible and real-time decision-making capabilities. By embedding domain-specific knowledge and leveraging LLM-driven reasoning, these systems can navigate complex production scenarios with greater autonomy and accuracy, ultimately improving the efficiency and adaptability of modern manufacturing processes. This approach has shown particular promise in integrating human and machine digital twins in dynamic environments to support real-time decision-making and training [34]. Furthermore, advanced methodologies such as graph-based neural networks are being explored to improve the diagnosis and maintenance of large-scale machinery [35], further supporting the role of LLMs in intelligent manufacturing.

This study proposes the Multi-Agent Scheduling Chain (MASC) framework to address key limitations in traditional Flexible Job Shop Scheduling Problems (FJSP), specifically the challenges of semantic understanding, decision-making, and adaptability in dynamic production environments. MASC leverages the decision-making capabilities of

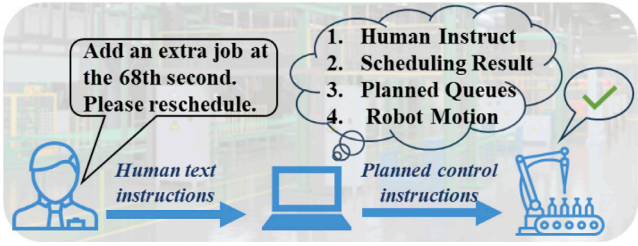


Fig. 1. Illustration of the MASC method solved the FJSP.

Large Language Models (LLMs) to enhance the semantic comprehension and operational efficiency in the scheduling process. While traditional FJSP solutions often rely on heuristic or optimization-based methods that demand significant domain expertise, MASC integrates LLMs with multi-agent technologies to improve flexibility and automation. By introducing LLMs, MASC enables natural language interaction and real-time adaptation to dynamic production requirements, which is critical for environments with frequent changes and complex workflows.

To our knowledge, the first attempt to integrate LLMs with MAS for solving FJSP is represented by this approach, making it both an original and timely contribution to the field of intelligent manufacturing.

The proposed framework includes several innovations to ensure that LLMs can effectively manage scheduling tasks. These innovations involve the construction of specialized datasets, the development of advanced decision-making algorithms, and the design of a comprehensive multi-agent system, as shown in Fig. 1.

The main contributions of this study are:

1. *SchedAgent Decision-Making Algorithm*: The SchedAgent (Scheduling Agent) was designed by enhancing the ReAct (Reason + Act) method to support more intelligent scheduling decisions. Specifically, SchedAgent integrates job-specific indicators that are critical for solving FJSPs, such as time dispersion, available machines, and operations. This method enables the agent to dynamically choose the optimal scheduling algorithm from a library of decision algorithms based on these indicators, ensuring that the solution aligns with real-time production requirements.

2. *Development of the DialBag Method*: The DialBag (Dialogue Bagging) method was developed to fine-tune the LLM for scheduling tasks by constructing datasets that incorporate single-turn dialogues, multi-turn dialogues, and case analyses. These datasets were used to distill the dialogue processes, enabling the model to retain domain-specific knowledge and improve its decision-making ability. This approach prevents knowledge forgetting, a common issue in LLM fine-tuning, and enhances the model's ability to interpret complex scheduling requirements accurately.

3. *Construction of the MASC Framework*: The MASC framework consists of four intelligent agents—ObsAgent (Observation Agent), SchedAgent, PlanAgent (Planning Agent), and CtrlAgent (Control Agent)—that collaborate to handle scheduling tasks in a cooperative and coordinated manner. ObsAgent is responsible for collecting workshop data and monitoring real-time production conditions, ensuring that the system remains aware of changes. SchedAgent dynamically selects and applies the most suitable scheduling algorithm from the tool library based on real-time production needs. PlanAgent transforms the scheduling results into actionable plans for machinery, aligning the scheduled tasks with downstream operations. Finally, CtrlAgent directly controls the robotic systems and other automated equipment to execute the planned tasks, ensuring a seamless flow in the production process. This multi-agent system enhances scalability and adaptability, making it well-suited for dynamic and complex workshop environments.

This study aims to significantly enhance the intelligence and efficiency of workshop scheduling by integrating LLMs into a multi-agent system. The proposed MASC framework offers a scalable solution for

dynamic, real-world production environments, validated through both simulations and real-machine experiments with robotic arms, confirming its effectiveness and adaptability in handling complex scheduling tasks.

The remainder of this paper is structured as follows: Section 2 details the methodology of the MASC framework, including the DialBag method and ReAct-based decision-making. Section 3 presents the simulation and experimental validation, and Section 4 concludes the study and discusses potential future research directions.

2. Methodology

2.1. Multi-agent scheduling chain

The complexity of the workshop environment presents significant challenges for current scheduling solutions, which typically use heuristic or optimization algorithms tailored to specific production needs. These solutions lack the ability to understand and process semantic information, limiting their adaptability to diverse scheduling demands. For instance, in traditional methods, scheduling requests such as "**Please provide the optimal solution for this scheduling problem within 30 s**" are typically translated into actionable parameters by manually extracting key scheduling attributes (e.g., number of jobs, machines, constraints) and configuring them into algorithm-specific input formats. This process has been considered inefficient due to its reliance on human expertise, lack of standardization, and inability to support real-time adaptation in dynamic production environments. This process is inefficient and does not facilitate natural, real-time interaction. By contrast, leveraging LLMs enables a more intuitive and efficient way to communicate scheduling instructions, allowing the model to directly interpret and act upon complex semantic requests.

Traditional methods for solving FJSP often require manual programming to control automated equipment post-scheduling, which is inefficient. To address these limitations, the MASC framework was proposed as a cooperative multi-agent system that enables semantic understanding, adaptive scheduling, and automated control in dynamic manufacturing environments. The MASC framework was designed to include four intelligent agents that collaborate in an ordered sequence to perceive the environment, interpret scheduling instructions, generate feasible plans, and execute them on physical equipment. Among them, SchedAgent was developed as the core reasoning unit, utilizing LLMs to analyze case characteristics and select appropriate scheduling algorithms. The detailed roles and coordination of the four agents within the MASC framework are illustrated in Fig. 2.

2.2. Multi-agent method based on LLM

Multi-agent systems (MAS) can be categorized based on their interaction goals into cooperative, competitive, mixed, and self-interested structures. Cooperative systems feature agents with aligned goals, aiming to maximize shared benefits through direct communication or indirect collaboration. Competitive systems involve agents with conflicting goals, where each agent aims to achieve its own benefit through game-theoretic or adversarial interactions. Mixed systems combine both cooperation and competition, where agents cooperate in some aspects while competing for resources in others. Self-interested systems are those where agents primarily pursue their own goals, often with passive collaboration facilitated through mechanisms like auctions or contracts.

In LLM-based MAS, agents typically communicate through natural language, enhancing intuitive human interaction. These systems can involve either cooperative or competitive interactions. Cooperative interactions, whether unordered or ordered, enable agents to work together efficiently. Ordered cooperation ensures that agents act sequentially, focusing on upstream information for high task completion rates. Competitive interactions use game theory principles to refine strategies through agent competition.

Our proposed system employs a cooperative structure with ordered cooperative interactions, optimizing communication and collaboration for workshop scheduling. This approach allows each agent to perform specific roles, ensuring efficient and flexible production processes. Each agent $A_i = \{P_i, C_i, B_i, S_i\}$ is defined by its profile, capabilities, behaviors, and skills. The message passing and synchronization between agents over time can be modeled by:

$$M_{sr}^{t+1} = m(A_s^t, A_r^t, M_{sr}^t, E_{sr}^t) \tag{1}$$

where M_{sr}^{t+1} is the message from sender agent A_s to receiver agent A_r at time $t+1$, and E_{sr}^t represents the environmental information at time t. The function m defines the rules for message transmission and updating. The variable t is defined as a discrete time step in the agent-based decision-making sequence. It is used to represent the iterative point at which inter-agent messages are exchanged and internal states are updated.

(1) ObsAgent A_O is responsible for collecting and integrating information from various sources to facilitate the effective functioning of downstream agents. It gathers data from the workshop and human inputs, extracting valuable information based on specific tasks and requirements, and passing it on to other agents. ObsAgent continuously monitors and processes real-time workshop data, such as operational statuses and machine conditions, ensuring timely responses to changes in the production environment.

The information collection process can be represented as:

$$M_{OS}^{t+1} = m(A_O^t, A_S^t, M_{OS}^t, E_{OS}^t) = P(I^t(\mathbf{S}, \mathbf{H}, \mathbf{D}), \mathcal{T}_O) \tag{2}$$

where M_{OS}^{t+1} represents the message from agent A_O to agent A_S at time $t+1$, P represents the process of ObsAgent. I^t includes information from sensors $\mathbf{S}$, human inputs $\mathbf{H}$, and documents $\mathbf{D}$. $\mathcal{T}_O$ represents the tool library of ObsAgent. It leverages the strengths of large models in summarizing and extracting key data, maintaining compatibility with visual and fault diagnosis modules, and enhancing system extensibility through continuous learning capabilities.

(2) SchedAgent A_S interprets semantic information, performs decision making, and selects suitable algorithms to solve scheduling problems. Decision making refers to the process in which the agent evaluates the current state of the system, the production requirements, and constraints, then selects the most appropriate scheduling algorithm or method to meet those objectives. This process is supported by a knowledge base and a tool library that enable the agent to optimize its choices based on historical data and real-time inputs, improving production efficiency and stability. SchedAgent parses and converts input data, considering production requirements, scheduling quality, and response times. It dynamically adapts to changing production environments by extracting key performance indicators and selecting appropriate algorithms. SchedAgent's extensive knowledge and algorithm toolset enable it to choose the most effective tools and methods for specific situations, optimizing decision models based on historical data and improving overall production efficiency and stability.

$$\mathfrak{D}_S^{t+1} = d(A_S^t, M_{OS}^t, S_S^t, \mathcal{T}_S, \mathcal{A}_S) \tag{3}$$

where $\mathfrak{D}_S^{t+1}$ represents the decision made at time $t+1$, the function d represents the decision making, M_{OS}^t is the message received from ObsAgent A_O, and S_S^t is the current state of SchedAgent A_S. Decision making involves selecting the best course of action based on the received data, current system state, and available toolset, optimizing the scheduling process by evaluating production constraints and goals. $\mathcal{T}_S$ represents the tool library of SchedAgent. $\mathcal{A}_S$ represents the scheduling algorithm library of SchedAgent. This agent's extensive knowledge base and algorithmic toolkit enable it to optimize decision models based on historical data, thereby improving overall production efficiency and stability.

(3) PlanAgent A_P is tasked with interpreting scheduling results, planning downstream tasks, and allocating automated equipment. It

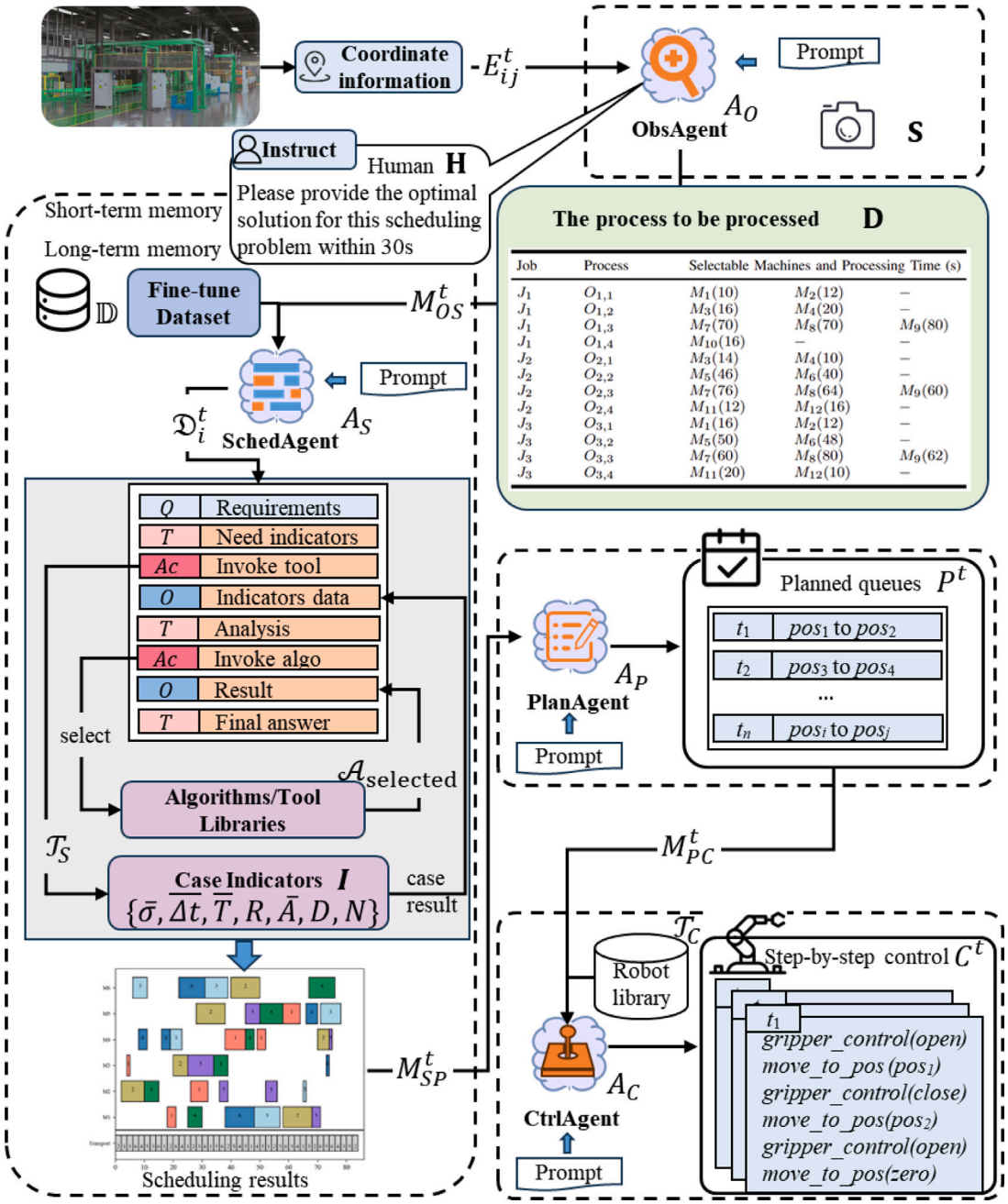


Fig. 2. Multi-agent scheduling chain framework. The MASC framework consists of four intelligent agents, each with distinct roles: **ObsAgent** A_O integrates workshop information and receives scheduling requests, **SchedAgent** A_S makes decisions based on integrated information, **PlanAgent** A_P plans downstream tasks and schedules automated equipment, and **CtrlAgent** A_C issues control commands to automated equipment. Additionally, a process re-encoding method, including transfer time, is introduced, treating the transfer process as a special machine integrated into the workflow for optimization. The MASC framework can analyze needs, solve scheduling problems, plan results, and control automated equipment, significantly enhancing efficiency and adaptability in workshop scheduling.

comprehensively understands scheduling outcomes, extracting and integrating task data using large models' semantic processing capabilities. PlanAgent assigns tasks to automated equipment with clearly defined start and end times, ensuring continuity and efficiency.

The task planning process can be formalized as:

$$P^t = k(A_P^t, M_{SP}^t, \mathcal{T}_P) \tag{4}$$

where P^t is the planned task at time t, the function k represents the planning, and M_{SP}^t is the message received from SchedAgent A_S. $\mathcal{T}_P$ represents the tool library of PlanAgent A_P. PlanAgent dynamically adjusts production plans based on real-time feedback, maintaining overall production efficiency and handling unforeseen events effectively to optimize task execution.

(4) CtrlAgent A_C converts planning instructions into executable commands for automated equipment. It accurately parses specific task instructions, including time, location, and actions, and generates corresponding control commands from its function library. The control command generation is expressed as:

$$C^t = n(A_C^t, M_{PC}^t, \mathcal{T}_C) \tag{5}$$

where C^t is the control command at time t, the function n represents control command generation, and M_{PC}^t is the message received from PlanAgent A_P. $\mathcal{T}_C$ represents the tool library of CtrlAgent A_C. CtrlAgent ensures the safety and accuracy of control commands by leveraging large models' tool-calling capabilities. Its modularity allows for the

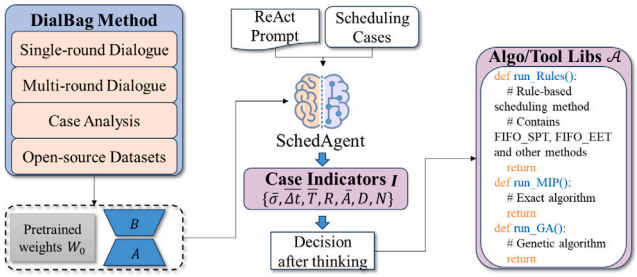


Fig. 3. Construction Process of the SchedAgent. The architectural flow of the scheduling agent is illustrated, where a fine-tuned large language model is integrated with a scheduling-specific tool library. The decision-making mechanism is driven by the improved ReAct method. Inputs are sequentially processed, and reasoning steps are performed to select algorithms and generate final scheduling decisions.

control of various equipment or tasks, adapting to diverse needs and ensuring precise and flexible operation of automated equipment.

2.3. Improved ReAct method for SchedAgent

The construction of the flexible job shop scheduling algorithm tool library $\mathcal{A}$ encompasses rule-based, exact, and heuristic methods to address different semantic contexts and problem scales. The large model algorithm leverages these tools for decision-making. Additionally, the model's self-awareness and professional knowledge are enhanced through single-round and multi-round dialogue distillation. Fig. 3 illustrates the construction method of the scheduling agent, SchedAgent, focusing on the algorithm library, instruction fine-tuning dataset, and the improved ReAct [36] method.

The improved ReAct-based SchedAgent algorithm (as shown in **Algorithm** 1) operates by integrating tool invocation into the agent's decision-making process to enhance scheduling solutions. Tool invocation refers to the process of selecting and utilizing algorithms from a predefined library to assist in solving scheduling problems. In this process, the agent calls upon relevant tools based on system inputs to aid in making informed decisions, optimizing the scheduling process. Initially, the algorithm sets the time step t to zero and initializes the context c_0 as an empty set. It then receives the initial observation o_0 from the environment based on the problem requirements Req, updating the context to include this observation.

At each iteration, the agent increments the time step and assesses whether it can make a direct decision or needs to invoke a tool from the toolset $\mathcal{T}$. If a direct decision is not feasible, the agent performs the following steps:

1. **Tool Invocation**: It invokes the metrics parsing tool from $\mathcal{T}$, represented as an action a_t, to obtain crucial indicators **I**. The context c_t is updated to include this action and the resulting observation o_t.

2. **Analysis and Algorithm Selection**: The agent analyzes the obtained indicators **I** along with the requirements Req to determine the most suitable algorithm $\mathcal{A}_{\text{selected}}$ from the algorithm library $\mathcal{A}$. This selection process can be represented by maximizing the conditional probability:

$$\mathcal{A}_{\text{selected}} = \arg\max_{\mathcal{A}_i \in \mathcal{A}} \text{Perf}(\mathcal{A}_i \mid \text{Req}) \tag{6}$$

where $\mathcal{A}_{\text{selected}}$ is the algorithm $\mathcal{A}_i$ that maximizes performance Perf given the requirement Req.

3. **Algorithm Execution**: The selected algorithm $\mathcal{A}_{\text{selected}}$ is executed, resulting in a new observation o_{t+1}. The context is further updated with this action and observation.

4. **Solution Assessment**: The agent checks if the result satisfies the problem requirements. If it does, the final solution is outputted, and the algorithm terminates.

If the agent determines that a direct decision can be made without tool invocation, it proceeds to make the decision, updates the context with the action and observation, and continues the loop.

Throughout the process, the agent leverages the ReAct framework by alternating between thought (*Thought*), action (*Action*), and observation (*Observation*) steps, effectively integrating external tools and context updates into its reasoning. This allows the agent to make informed decisions based on both immediate observations and derived metrics, enhancing its problem-solving capabilities in flexible job shop scheduling scenarios.

The improved ReAct method integrates seven indicators $\mathbf{I} = \{\bar{\sigma}, \overline{\Delta t}, \overline{T}, R, \bar{A}, D, N\}$ to provide the agent with a comprehensive understanding of scheduling cases, enhancing decision-making capabilities.

(1) *Time Dispersion*($\bar{\sigma}$): Calculate the standard deviation of processing times for each job across all machines to assess time volatility. Larger dispersion suggests suitability for algorithms prioritizing high time variability.

$$\bar{\sigma} = \frac{\sum_{i=1}^{n_j} \sum_{j=1}^{n_m} \left(\sqrt{\frac{\sum_{k=1}^{n_{ij}} (t_{ijk} - \frac{\sum_{k=1}^{n_{ij}} t_{ijk}}{n_{ij}})^2}{n_{ij} - 1}} \right)}{n_j n_m} \tag{7}$$

where n_j is the total number of jobs, n_m is the total number of machines, t_{ijk} is the kth processing time of the ith job on the jth machine, and n_{ij} is the number of times the ith job is processed on the jth machine.

(2) *Time Difference*($\overline{\Delta t}$): Calculate the difference between the maximum $\max(t_{ij})$ and minimum processing times $\min(t_{ij})$ for all jobs on each machine to identify potential bottlenecks.

$$\overline{\Delta t} = \frac{1}{n_m} \sum_{j=1}^{n_m} (\max(t_{ij}) - \min(t_{ij})) \tag{8}$$

where t_{ij} is the processing time of ith job on the jth machine.

(3) *Average Time* ($\overline{T}$): Evaluate the average processing time for all jobs on all machines to gauge overall workload.

$$\overline{T} = \frac{1}{n_j n_m} \sum_{i=1}^{n_j} \sum_{j=1}^{n_m} t_{ij} \tag{9}$$

(4) *Job/Machine Ratio*(R): Reflect the ratio of jobs to machines, aiding in assessing scheduling complexity.

$$R = \frac{n_j}{n_m} \tag{10}$$

(5) *Available Machines*($\bar{A}$): Measure the average number of available machines per operation, indicating flexibility.

$$\bar{A} = \frac{1}{n_j} \sum_{i=1}^{n_j} \left(\frac{1}{n_i} \sum_{j=1}^{n_i} m_{ij} \right) \tag{11}$$

where m_{ij} is the ith job that can be processed on the jth machine, n_i is the number of steps for the ith job.

(6) *Job-Machine Matrix Density*(D): Calculate the density of the Job-machine matrix, representing the proportion of non-zero elements, indicating problem complexity.

$$D = \frac{\sum_{i=1}^{n_j} \sum_{j=1}^{n_m} sgn(t_{ij})}{n_j n_m} \tag{12}$$

where $sgn(t_{ij})$ is an indicator function that takes the value 1 when the ith job can be processed on the ith machine.

(7) *Operations* (N): Assess the total number of operations across all jobs to understand task complexity.

$$N = \sum_{i=1}^{n_j} n_i \tag{13}$$

By using these indicators, the agent's thinking process is refined, allowing for better tool-calling and more informed decisions. The instruction fine-tuning dataset, constructed using the DialBag method, enhances the model's pretrained weights by adding new weights tailored

Algorithm 1 SchedAgent Algorithm

Input: Problem requirements Req;
Toolset $\mathcal{T}$ (including metrics parsing tool);
Algorithm library $\mathcal{A}$ (set of algorithms).
Output: Final solution.

Initialize time step $t \leftarrow 0$;
Initialize context $c_0 \leftarrow \emptyset$;
Receive initial observation o_0 from the environment based on Req;
Update context $c_0 \leftarrow c_0 \cup \{o_0\}$;
while not solved **do**
  $t \leftarrow t + 1$;
  /* `Thought: Determine the necessity of tool invocation` */
  if Decision cannot be made directly **then**
    /* `Action: Invoke the metrics parsing tool from the toolset` $\mathcal{T}$ */
    $a_t \leftarrow$ InvokeTool(MetricsParsingTool, c_{t-1});
    /* `Observation: Obtain indicators` **I** */
    $o_t \leftarrow$ GetObservation(a_t);
    Update context $c_t \leftarrow c_{t-1} \cup \{a_t, o_t\}$;
    /* `Thought: Analyze indicators` **I** `and requirements` Req */
    Determine the appropriate algorithm $\mathcal{A}_{\text{selected}}$ based on **I** and Req;
    /* `Action: Select algorithm` $\mathcal{A}_{\text{selected}}$ `from the algorithm library` $\mathcal{A}$ */
    $a_{t+1} \leftarrow \mathcal{A}_{\text{selected}}$;
    /* `Observation: Execute algorithm` $\mathcal{A}_{\text{selected}}$ `and obtain the result` */
    $o_{t+1} \leftarrow$ ExecuteAlgorithm(a_{t+1});
    Update context $c_{t+1} \leftarrow c_t \cup \{a_{t+1}, o_{t+1}\}$;
    /* `Thought: Derive the final solution based on the results` */
    if Result meets the requirements **then**
      Output the final solution;
      return Final solution;
    end if
  else
    /* `Directly make a decision without invoking tools` */
    $a_t \leftarrow$ DirectDecision(c_{t-1});
    $o_t \leftarrow$ GetObservation(a_t);
    Update context $c_t \leftarrow c_{t-1} \cup \{a_t, o_t\}$;
  end if
end while

for scheduling algorithms. These tailored weights W (linear layer B and linear layer A) adapt the base LLM to specific scheduling scenarios [28].

At time t, the SchedAgent receives an observation sample $o^t \in \mathcal{O}$ from the environment. Based on the observation o^t, SchedAgent follows a policy $\pi(a^t|c^t)$ to take an algorithm $a^t \in \mathcal{A}$, where $c^t = (o^1, a^1, \ldots, o^{t-1}, a^{t-1}, o^t, a^t)$ is the context of the agent.

In our method, the SchedAgent can use the ReAct method to select tools from the toolset $\mathcal{T}$. At time t, the agent's observation samples and context are c^t, the scheduling case to be solved is $\mathcal{D}$, and the action space $\hat{A} = \mathcal{A} \cup \mathcal{T} \cup c^t$ contains the toolset $\mathcal{T}$ and the algorithms in the algorithm library $\mathcal{A}$. When the SchedAgent receives a solution request, it first calls the case analysis tool $\hat{a}^t \in \mathcal{T}$ from the toolset to obtain the observed value of the indicators **I**.

$$\mathbf{I} = f(o^t, \mathcal{T}, \mathcal{D}) = \{\bar{\sigma}, \overline{\Delta t}, \overline{T}, R, \bar{A}, D, N\} \tag{14}$$

where the seven indicators $\{\bar{\sigma}, \overline{\Delta t}, \overline{T}, R, \bar{A}, D, N\}$ are calculated by the observation o^t, the toolset $\mathcal{T}$, and the scheduling case to be solved $\mathcal{D}$.

After obtaining the indicators **I**, the agent selects the algorithm based on the context and the indicators. The selection probability of strategy π can be expressed as:

$$\pi(\mathcal{A}_i|c^t, \mathbf{I}, D) = \frac{\exp(Q(c^t, \mathbf{I}, \mathcal{A}_i, D))}{\sum_j \exp(Q(c^t, \mathbf{I}, \mathcal{A}_j, D))} \tag{15}$$

where $Q(c^t, \mathbf{I}, \mathcal{A}_i, D)$ represents the decision function based on the context c^t, the indicators **I**, and the case D.

The action selected according to strategy π is the algorithm $a^t = \mathcal{A}_i$. Then the context c^{t+1} can be updated as $c^{t+1} = (c^t, \mathbf{I}, \mathcal{A}_i, f(\mathcal{A}_i, D))$. In summary, the agent can utilize the toolset $\mathcal{T}$ to obtain key indicators **I**, and based on these indices and the case to be solved $\mathcal{D}$, analyze and reason, select the appropriate algorithm $\mathcal{A}_i$ for solving, and finally derive the solution to the problem.

2.4. Dialogue bagging (DialBag) method

To locally train an LLM with scheduling knowledge, a high-quality dialogue corpus is necessary for knowledge alignment. Inspired by Ho et al.'s approach [37], this study employs larger models (e.g., GPT-4 [13]) to guide smaller models through single/multi-round dialogue distillation, extracting reasoning capabilities from larger models to enhance local models.

As shown in Fig. 4. In single-round distillation, questions are randomly combined and input into large models for dialogue distillation, collecting responses from GPT-4 [13], QWEN [38], and GLM-4 [39]. This ensures diverse, high-quality data. The process can be represented as follows:

$$\mathbb{D}_{\text{single}} = \{(q, f_{\mathbf{M}}(q)) \mid q \in Q, \mathbf{M} \in \{\mathbf{M}_1, \mathbf{M}_2, \mathbf{M}_3\}\} \tag{16}$$

where $\mathbb{D}_{\text{single}}$ denotes the dataset collected from single-round distillation, Q is the set of questions, and $f_{\mathbf{M}}$ represents the response function of the models GPT-4, QWEN, and GLM-4, which represented by $\mathbf{M}_1$, $\mathbf{M}_2$, and $\mathbf{M}_3$.

In multi-round distillation, a more complex mechanism simulates real dialogue environments, with one LLM as an expert and another as an evaluator, ensuring comprehensive and coherent dialogue. This

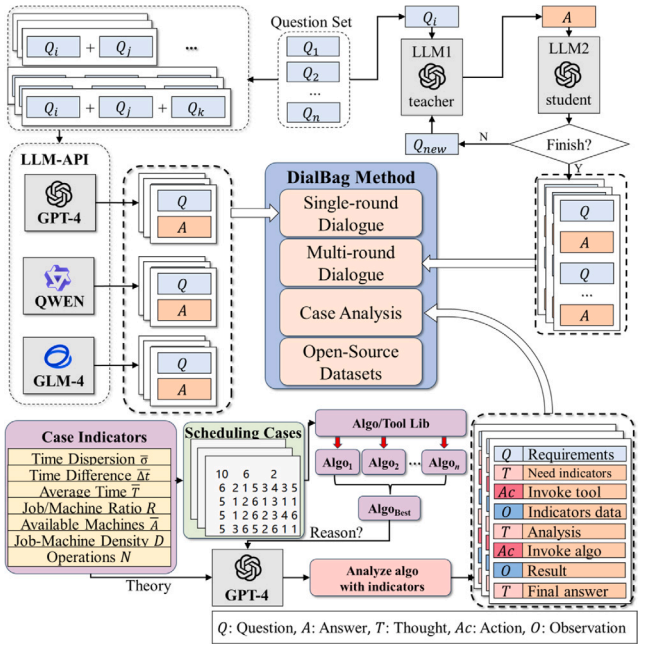


Fig. 4. The Proposed Dialogue Bagging (DialBag) Method. The figure illustrates the DialBag method, which is composed of four distinct but interconnected processes. The Single/Multi-round Dialogue Distillation process, represented in the top row, is utilized to distill knowledge from other models to enhance the performance of the target model. The Case Analysis Process, shown in the bottom row, is designed to extract and analyze the decision-making steps of the model.

method maximizes valuable data extraction through iterative questioning and evaluation. The process is represented by:

$$\mathbb{D}_{\text{multi}} = \begin{cases} (q_i, f_{\mathbf{E}}(q_i) \mid q_i \in Q), & \text{if } i = 1 \\ (h_i, f_{\mathbf{E}}(h_i, q_i) \mid q_i = f_{\mathbf{V}}(h_i)), & \text{if } i > 1 \end{cases} \tag{17}$$

where $h_i = (q_1, a_1, \ldots, q_{i-1}, a_{i-1})$ denotes the dialogue history up to the ith turn, composed of previous questions q and answers a. The functions $f_{\mathbf{E}}$ and $f_{\mathbf{V}}$ represent the response generation by the expert model and the question generation by the evaluator model, respectively.

An indicator analysis dataset for workshop scheduling cases is constructed by calculating various indicators, selecting the best algorithm based on requirements, and generating a reasoning trajectory using GPT-4. These indicators $\mathbf{I}$ for the given scheduling case $\{\bar{\sigma}, \bar{\Delta t}, \bar{T}, R, \bar{A}, D, N\}$ serve as the theoretical foundation. The case indicator analysis dataset can be represented as follows:

$$\mathbb{D}_{\text{analysis}} = \{(\mathbf{I}, f_{\mathbf{GPT}}(\mathbf{I}, \mathcal{A}_{\text{selected}}), \mathcal{A}_{\text{selected}})\} \tag{18}$$

where $\mathcal{A}_{\text{selected}} = \arg\max_{\mathcal{A}_i \in \mathcal{A}} \text{Perf}(\mathcal{A}_i \mid \text{Req})$ is the algorithm that maximizes performance Perf given the requirement Req. The function $f_{\mathbf{GPT}}$ represents the GPT-4 model used to output the reasoning trajectory.

2.5. Large model agent training method

In the proposed methodology for training large language models, multiple data distributions are incorporated to ensure comprehensive learning and robust performance. The objective function $J(\theta)$ for the training process is defined as follows:

$$J(\theta) = \sum_{k \in \{\text{s,m,a,o}\}} \eta_k \cdot \mathbb{E}_{(x,y)\sim\mathbb{D}_k} \left[\log \pi_\theta(y \mid x)\right] \tag{19}$$

where $k \in \{\text{s, m, a, o}\}$ represents the $\mathbb{D}_{\text{single}}$, $\mathbb{D}_{\text{multi}}$, $\mathbb{D}_{\text{analysis}}$, and $\mathbb{D}_{\text{open}}$ datasets respectively. Each dataset is assigned a weight parameter η_k to balance its contribution to the overall objective. These weights reflect the relative importance or the amount of data available from each distribution.

Table 1
Dataset overview [40].

Dataset	Total count	Sample count	Sampling method
Single/Multi-Round Distillation	1735	5000	Repeated
Improved ReAct	869	3000	Repeated
Ms-agent	30,000	5000	Random
Ms-bench	380,000	5000	Random
Final Dataset	–	18,000	–

To efficiently fine-tune our LLM, QLoRA [29] is employed, an advanced technique that optimizes only a small portion of the model parameters. QLoRA is built upon LoRA [28], with small, trainable parameters (Adapters) introduced alongside pre-trained ones. This approach is designed to reduce training time and cost without sacrificing accuracy, and its parallel structure ensures that the original model outputs remain unchanged while the model's expertise for specific tasks is enhanced.

3. Experiments

3.1. Agent training configuration

Table 1 summarizes the dataset composition, including single-round and multi-round dialogue distillation, improved ReAct, and open datasets (Ms-agent and Ms-bench) [40], ensuring a balanced mix of high-quality data for fine-tuning.

Considering the need for low-cost fine-tuning and deployment while ensuring inference accuracy, this study selects a 7B parameter model as the base model for fine-tuning. Given that Internlm2-7B [41] demonstrates strong performance on benchmark datasets among 7B models, it is chosen as the base model for this study to maintain or enhance performance on specific tasks. Training Configuration: Linux 5.10, CPU: Intel(R) Xeon(R) Platinum 8369B @2.90 GHz, GPU: 1/4 A100, VRAM: 24 GB.

3.2. SchedAgent approach verification

As shown in Fig. 5, the model's cognitive abilities and problem-solving capabilities are enhanced post fine-tuning.

Due to dataset limitations, only partial validation results are presented. Experiments were conducted using well-known public datasets, including instances from Barnes, Brandimarte, Dauzere, and Hurink, referred to as BCdata [42], BRdata [43], DPdata [44], and HUdata [45], respectively. HUdata is further divided into four subsets: edata, rdata, sdata, and vdata. For a comprehensive evaluation, the overall performance is averaged across all seven datasets. Ten comparison methods were employed to solve the FJSP: Genetic Algorithm (GA), Mixed Integer Programming (MIP), and eight rule-based methods. Given the large number of rule-based algorithms, only the one with the highest overall ranking percentage—First In First Out (FIFO) combined with Earliest End Time (EET)—is presented in the comparison charts. By using these datasets and comparison methods, this study aims to ensure a comprehensive evaluation of the proposed algorithm.

Since no single algorithm performs optimally in every case, a ranking mechanism was designed to further assess the generality of SchedAgent. A higher ranking percentage indicates that the method yields better solutions for a given case. The ranking percentage p_{ij} for method j in case i is defined as:

$$p_{ij} = \left(1 - \frac{r_{ij} - 1}{m - 1}\right) \times 100\%, \tag{20}$$

where r_{ij} represents the rank of method j in case i, and m is the total number of algorithms in the library. Furthermore, the average ranking percentage $\bar{p}_j$ for method j is calculated as:

$$\bar{p}_j = \frac{\sum_{i=1}^{n} p_{ij}}{n}, \tag{21}$$

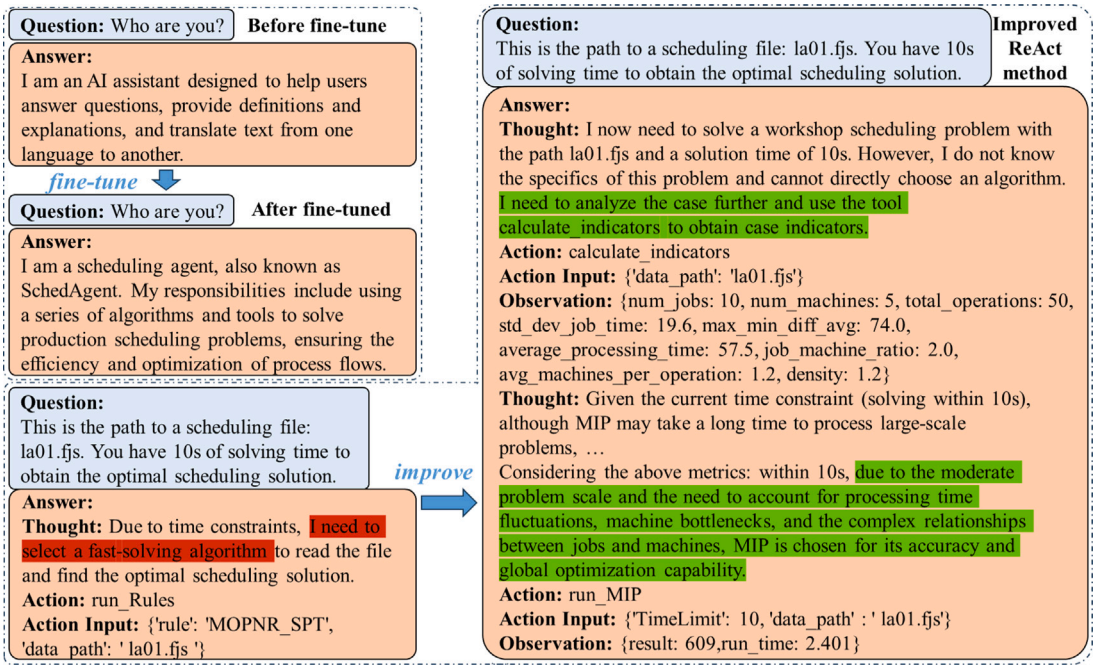


Fig. 5. Comparison before and after fine-tuning. The improvement in SchedAgent's cognitive understanding and problem-solving capabilities after fine-tuning is shown in the figure. A better understanding of its identity and responsibilities was gained by SchedAgent, and its analytical thinking and decision-making skills were enhanced with the improved ReAct method [36].

where n represents the total number of cases in the dataset, the average ranking percentage $\bar{p}_j$ for each dataset was computed to quantitatively compare the performance of SchedAgent with other algorithms under specific time constraints. A higher $\bar{p}_j$ value, expressed as a percentage, indicated that algorithm j achieved better solutions for the dataset under the given conditions.

Given a 10 s solution time, the benchmark dataset problems were solved, and SchedAgent was provided with prompt inputs specifying the solution time and problem path, as shown in Fig. 6, demonstrating that when a quick solution is required, the MIP model struggles with large datasets, whereas rule-based scheduling methods become more effective.

The validation results indicate that SchedAgent consistently ranks highly in percentage rankings across various datasets, demonstrating excellent performance. By analyzing problem requirements and characteristics, SchedAgent effectively decides on appropriate solution methods. This adaptability allows SchedAgent to perform well in both quick solution scenarios and more complex scheduling environments, demonstrating its versatility and efficiency.

Given a 30 s solution time, the benchmark dataset problems are solved, and SchedAgent is provided with prompt inputs specifying the solution time and problem path. The results are shown in Fig. 7.

The experimental results show that with a relaxed 30 s solution time, the MIP model becomes the optimal choice in some datasets. SchedAgent outperforms single models in understanding scheduling problem semantics and making decisions. Some cases show better results in shorter times due to random early convergence.

The results show that with a 10 s solution time, the MIP model consistently achieves the best results in the BCdata dataset, while in the DPdata dataset, it performs the worst, with rule-based methods quickly providing results and achieving the best ranking percentage. The ranking percentage was calculated based on the method's rank among all tested algorithms, with the highest-ranking method receiving the highest rate. This metric helped quantify each method's effectiveness in solving the scheduling problems, showing that SchedAgent demonstrated significant improvements and good adaptability across

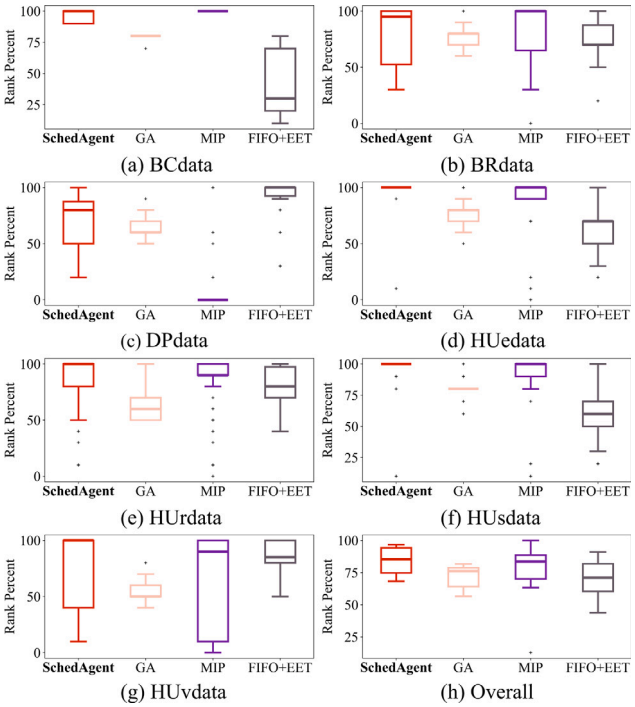


Fig. 6. Ranking percentage with a 10 s solving time. The statistical distribution of SchedAgent's performance compared with other algorithms is shown in this figure. A higher median and narrower interquartile range are interpreted as indicators of more consistent and effective scheduling outcomes.

different datasets. With a 30 s solution time, the performance of various algorithms is similar across different datasets, but the ranking percentage of exact algorithms improves significantly compared to the 10 s results. SchedAgent outperforms other algorithms overall, with a

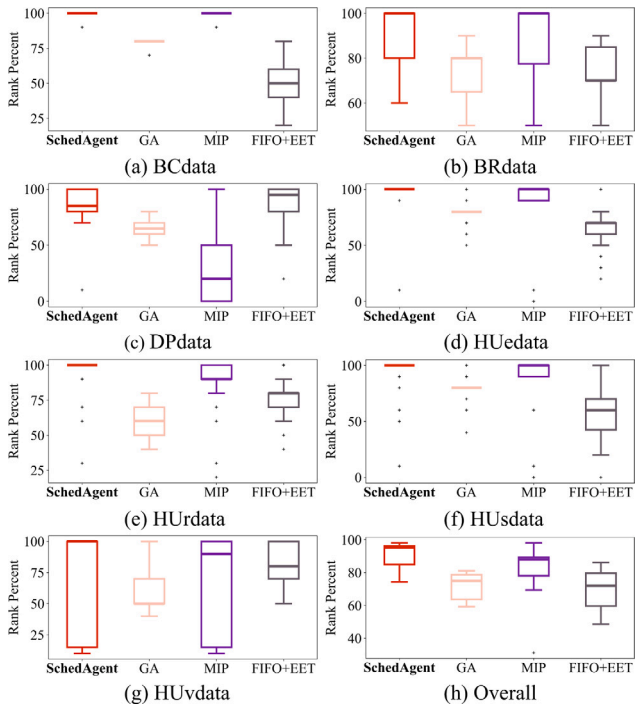


Fig. 7. Ranking percentage with a 30 s solving time. SchedAgent's improved performance under relaxed time constraints is demonstrated in this figure, and its adaptability and competitiveness against both exact and heuristic scheduling methods are highlighted.

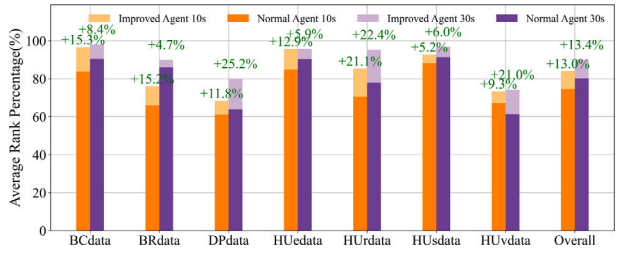


Fig. 8. Comparison before and after improvement for Internlm2-7B [41].

total ranking percentage of 84% under a 10 s solution time and 90% under a 30 s solution time. This indicates a significant improvement in decision-making capabilities under current semantic conditions, with SchedAgent demonstrating good adaptability across different datasets.

The effectiveness of the improved ReAct method in decision-making for scheduling algorithms is validated by comparing the Internlm2-7B model using the improved ReAct method with solution times of 10 s and 30 s, as shown in Fig. 8 . With a 10 s solution time, the total ranking percentage of Internlm2-7B increased by 13%. When extended to a 30 s solution time, Internlm2-7B showed a further improvement, with the total ranking percentage increasing by 13.4%. This demonstrates the significant enhancement in decision-making efficiency and effectiveness using the improved ReAct method.

Similarly, the Llama3-8B model was evaluated using the improved ReAct method. With a 10 s solution time, Llama3-8B exhibited a remarkable improvement, with the total ranking percentage increasing by 63.1%, as shown in Fig. 9. This significant gain is attributed to its initial lack of pre-trained Chinese enhancement. With a 30 s solution time, Llama3-8B showed an improvement of 19.1% across all datasets. However, it experienced an 8% decrease in the DPdata dataset, likely due to the better performance of rule-based scheduling methods and the insufficient representation of similar data in the improved ReAct dataset, leading to data bias.

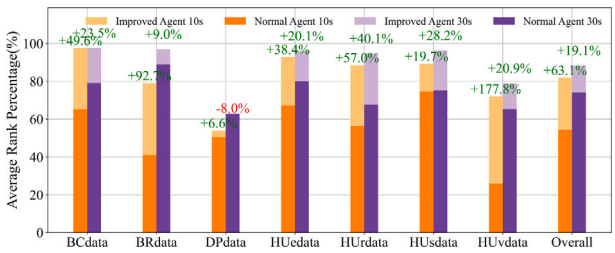


Fig. 9. Comparison before and after improvement for Llama3-8B [46].

Table 2
Performance comparison of the DialBag method across different datasets.

Method	ARC	HellaSwag	MMLU	SchedQA
internlm2-7b	83.73	85.64	**62.19**	31.36
internlm2-7b + single round	72.2	81.01	61.01	31.47
internlm2-7b + DialBag	**84.75**	**86.14**	58.93	**32.11**

3.3. Experimental validation of the DialBag method

The effectiveness of the DialBag method is demonstrated through comparative experiments, as shown in Table 2.

ARC [47], HellaSwag [48], and MMLU [49] are commonly used datasets for evaluating LLMs. SchedQA represents the collected knowledge questions and answers in the scheduling field. The "single round" method refers to the fine-tuning results obtained using a common single-round dialogue dataset, where performance improvements on SchedQA were observed, but some knowledge was forgotten. In contrast, the "DialBag" method, which uses a mixed dataset, resulted in less knowledge forgetting across other areas and showed improved performance on SchedQA. It was found that the proposed DialBag method successfully mitigates knowledge forgetting and improves the incorporation of domain-specific knowledge. However, the performance of the internlm2-7b+DialBag method on the MMLU dataset was lower, which may be attributed to the insufficient richness of the dataset, causing some knowledge to be forgotten even after QLoRA fine-tuning.

3.4. MASC system experimental verification

The algorithm library configured in MASC is shown in Table 3. The libraries referenced by the agents, contain a variety of algorithms that can be adjusted or expanded based on the scheduling problem type. These libraries are not limited to FJSP, they can be adapted to other production scenarios depending on the system constraints and the semantic understanding required by the agents. Each agent selects the corresponding tool function based on system prompts and passes the appropriate parameters. The libraries used by the different agents consist of a range of scheduling and decision-making algorithms. ObsAgent's library contains algorithms for data collection and monitoring, SchedAgent uses algorithms from the decision library (such as GA and MIP), which are selected dynamically based on real-time production data, performance indicators, and system constraints. The selection process involves analyzing production requirements such as time dispersion, available machines, and task complexity. The correctness and rationality of tool invocation by the agent directly affect the algorithm's success rate. In dynamic environments, where production conditions frequently change, SchedAgent's algorithm selection process adapts by integrating reinforcement learning (RL) techniques to further optimize decision-making. RL methods complement traditional scheduling algorithms, enabling SchedAgent to make more flexible and accurate decisions in the face of uncertainty, while maintaining the efficiency of established heuristics like GA and MIP. Additionally, new algorithms and methods can be added as needed for specific manufacturing processes, making the framework flexible.

Table 3
Multi-agent algorithm tool library configuration.

Agent	Algorithm tool function	Meaning
ObsAgent	*get_coordinate*	Get coordinate information
SchedAgent	*calculate_indicators*	Calculate various performance indicators
SchedAgent	*run_GA*	Run genetic algorithm to solve scheduling
SchedAgent	*run_MIP*	Run mixed-integer programming algorithm
SchedAgent	*run_Rules*	Execute rule-based scheduling
SchedAgent	*update_process_robots*	Add automated equipment to process flow
SchedAgent	*update_process_fault*	Change process flow due to equipment fault
SchedAgent	*update_process_order*	Change process flow due to urgent orders
PlanAgent	*decode_robot_transfers*	Parse start time and starting point of transfers
CtrlAgent	*move_to_position*	Change process flow due to urgent orders
CtrlAgent	*gripper_control*	Move robotic arm to specified coordinates

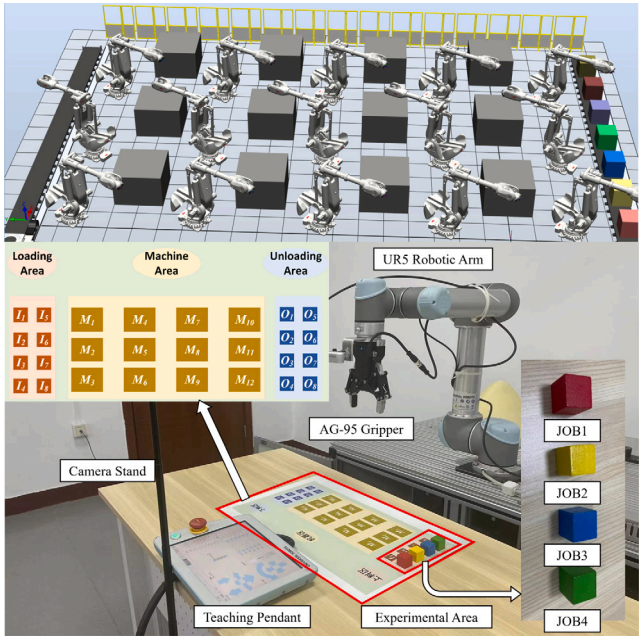


Fig. 10. Simulation (upper) and Experimental Platform (lower). Robotstudio was used to simulate the scenario. The system's resilience and rescheduling capabilities under different conditions were tested using equipment failures and emergency orders. The experimental setup included a UR5 robot and an AG-95 gripper for job handling and transfer.

RobotStudio was used to simulate a scheduling scenario involving multiple workpieces and 12 different types of machines. To replicate potential issues encountered during the workpiece transfer process in a real industrial environment, a UR5 robot was employed to transport the workpieces. The staying time of a workpiece in a specific area was used to indicate that a certain process was being performed on the corresponding machine. The configuration of the simulation and real machine experiment is shown in Fig. 10.

The experiment was set up in three sections: the loading area, the machine area, and the unloading area. The loading area was designated to store the workpieces to be scheduled, the machine area was responsible for processing the workpieces, and the unloading area was used to store the workpieces after the scheduling process. A total of eight loading areas, 12 machine areas, and eight unloading areas were configured. The workpieces were initially placed in the loading area, and a manipulator transported the workpieces to the corresponding machines for processing. After the processing was completed, the workpieces were transferred to the unloading area. The processing time of each workpiece varied depending on the machine. During the experiment, the workpieces followed the relevant process flow, with each flow having multiple optional machines, each with different processing times. The machines and processing times for each job and process are detailed in Table 4.

Table 4
Details of the job and process flow, including machine selection and processing times.

Job	Process	Selectable machines and processing time (s)		
J_1	$O_{1,1}$	$M_1(10)$	$M_2(12)$	–
J_1	$O_{1,2}$	$M_3(16)$	$M_4(20)$	–
J_1	$O_{1,3}$	$M_7(70)$	$M_8(70)$	$M_9(80)$
J_1	$O_{1,4}$	$M_{10}(16)$	–	–
J_2	$O_{2,1}$	$M_3(14)$	$M_4(10)$	–
J_2	$O_{2,2}$	$M_5(46)$	$M_6(40)$	–
J_2	$O_{2,3}$	$M_7(76)$	$M_8(64)$	$M_9(60)$
J_2	$O_{2,4}$	$M_{11}(12)$	$M_{12}(16)$	–
J_3	$O_{3,1}$	$M_1(16)$	$M_2(12)$	–
J_3	$O_{3,2}$	$M_5(50)$	$M_6(48)$	–
J_3	$O_{3,3}$	$M_7(60)$	$M_8(80)$	$M_9(62)$
J_3	$O_{3,4}$	$M_{11}(20)$	$M_{12}(10)$	–

Table 5
The machine selection and processing times after equipment failure, reflecting the task reallocation and rescheduling.

Job	Process	Selectable machines and processing time (s)		
J_1	$O_{1,1}$	$M_1(10)$	$M_2(12)$	–
J_1	$O_{1,2}$	$M_4(20)$	–	–
J_1	$O_{1,3}$	$M_7(70)$	$M_8(70)$	$M_9(80)$
J_1	$O_{1,4}$	$M_{10}(16)$	–	–
J_2	$O_{2,2}$	$M_5(46)$	$M_6(40)$	–
J_2	$O_{2,3}$	$M_7(76)$	$M_8(64)$	$M_9(60)$
J_2	$O_{2,4}$	$M_{11}(12)$	$M_{12}(16)$	–
J_3	$O_{3,2}$	$M_5(50)$	$M_6(48)$	–
J_3	$O_{3,3}$	$M_7(60)$	$M_8(80)$	$M_9(62)$
J_3	$O_{3,4}$	$M_{11}(20)$	$M_{12}(10)$	–

For unexpected events, such as machine breakdowns or urgent job additions, MASC employs event-driven rescheduling through the ObsAgent, which detects changes in the system and signals SchedAgent to revise the schedule. The SchedAgent then uses real-time data to adapt the existing schedule, ensuring minimal disruption to production. Rescheduling is carried out based on predefined constraints and objectives, prioritizing jobs that require immediate attention.

To demonstrate MASC's feasibility in a dynamic scheduling environment, an equipment failure scenario is considered. In the first experiment, one piece of equipment(Machine 3) suddenly fails, requiring rescheduling. The disturbance is introduced at the 20th second, with the prompt: "**Machine 3 has malfunctioned at the 20th second. Please reschedule.**" This prompt caused a breakdown in Machine 3, requiring the SchedAgent to reschedule the jobs and reallocate them to available machines in real-time. The experiment evaluates MASC's ability to quickly respond to malfunctions and ensure minimal disruption to the production flow. MASC removes Machine 3 and generates a new process flow, as shown in Table 5.

The real machine control results for equipment failure, indicating that MASC successfully rescheduled and executed control commands, completing the rescheduling process. The TCP (Tool Center Point) variation curve for the robotic arm under equipment failure is shown in Fig. 11.

To further validate, an urgent order scenario is considered. The disturbance is introduced at the 68th second, with the prompt: "**Add an extra job at the 68th second. Please reschedule.**" This new job was introduced as an additional order requiring immediate integration into the existing production schedule. The SchedAgent dynamically adjusted the schedule by assigning machines and reordering the job sequences to

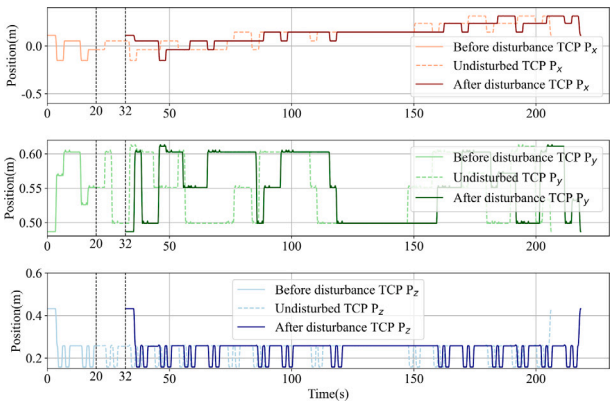


Fig. 11. TCP Variation Curve under Equipment Failure. The fluctuation in the TCP of the robotic arm during an equipment failure is shown in this figure. It highlights the system's ability to adapt in real-time, maintaining stability and adjusting its operations to mitigate the impact of the failure on the production process. The curve reflects the variation in TCP position as a result of the equipment failure, illustrating the dynamic response of the robotic arm.

Table 6
The machine selection and processing times after the addition of an urgent order, reflecting the task adjustment and reallocation.

Job	Process	Selectable machines and processing time (s)		
J_1	$O_{1,3}$	$M_7(60)$	$M_8(70)$	$M_9(80)$
J_1	$O_{1,4}$	$M_{10}(16)$	–	–
J_2	$O_{2,3}$	$M_7(76)$	$M_8(64)$	$M_9(60)$
J_2	$O_{2,4}$	$M_{11}(12)$	$M_{12}(16)$	–
J_3	$O_{3,3}$	$M_7(60)$	$M_8(80)$	$M_9(62)$
J_3	$O_{3,4}$	$M_{11}(20)$	$M_{12}(10)$	–
J_4	$O_{1,1}$	$M_1(10)$	$M_2(14)$	–
J_4	$O_{1,2}$	$M_5(40)$	$M_6(50)$	–
J_4	$O_{1,3}$	$M_{11}(12)$	$M_{12}(11)$	–

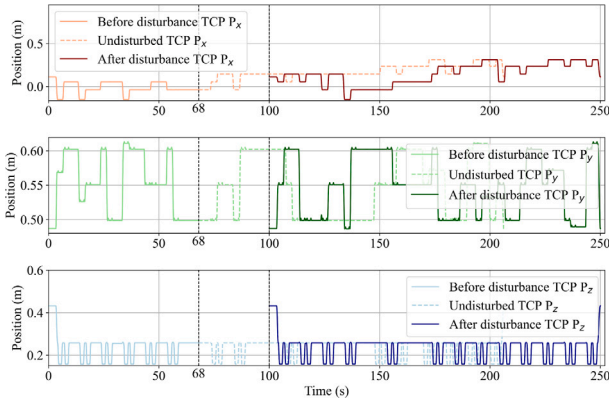


Fig. 12. TCP Variation Curve under Urgent Orders. The figure illustrates how the TCP of the robotic arm varies when an urgent job is added to the production schedule. It demonstrates the system's flexibility in adapting to new tasks while maintaining operational efficiency. The curve showcases the robotic arm's ability to adjust its movements and scheduling priorities in real-time, responding to the unexpected scheduling changes with minimal disruption to ongoing processes.

accommodate the new task without significantly delaying other jobs in progress. MASC evaluates the current state and generates a new process flow, as shown in Table 6.

The real machine control results for urgent orders indicate that MASC successfully rescheduled and executed control commands, completing the rescheduling process. The TCP variation curve for the robotic arm under urgent orders is shown in Fig. 12.

4. Conclusion

The complexity of FJSP presents significant challenges due to the need for effective resource allocation, process optimization, and automation integration. To address these issues, the MASC framework has been proposed, integrating LLMs and agent technologies, specifically utilizing a 7B model fine-tuned with QLoRA, to develop the scheduling agent SchedAgent. This approach enhances tool invocation capabilities and mitigates hallucination issues, providing a comprehensive solution for intelligent scheduling and control.

Quantitative results indicate that with a 10 s solving time requirement, SchedAgent achieves an overall ranking percentile of 84% by selecting the appropriate algorithm. With a 30 s solving time requirement, the overall ranking percentile improves to 90%, significantly outperforming other algorithms. This underscores the advantages of integrating LLM into the scheduling process, demonstrating enhanced decision-making capabilities and adaptability across different datasets.

MASC has been experimentally validated using both a robotic arm and simulation. In both setups, the four agents coordinated the scheduling process and completed automated transfer utilizing the configured tool library. The experiments have confirmed the effectiveness and stability of MASC in dynamic scheduling environments.

These conclusions validate MASC's core innovations and methodologies, highlighting its potential to transform workshop scheduling. By leveraging LLMs for decision-making, MASC overcomes traditional FJSP limitations, enhancing production efficiency and flexibility.

Although the adoption of a large language model introduces concerns regarding inference latency, the feasibility and effectiveness of LLM-based decision-making have been primarily investigated in this work. A 7B parameter model fine-tuned with QLoRA was selected to strike a balance between performance and inference cost. Empirical evaluations showed that the average response time of the current model is approximately 20 s, which can be considered near real-time. It is acknowledged that the inference latency is significantly affected by model size and computing hardware. Therefore, with the deployment of higher-performance computing resources, it is expected that second-level real-time response can be achieved.

Future work could involve improving agent collaboration and integrating advanced machine learning techniques for better decision-making. Expanding MASC to accommodate more diverse and complex production environments will also be pursued. Graph-based reasoning can optimize scheduling, hybrid digital twins can enable real-time adaptive scheduling, and ontology-based knowledge representation can enhance semantic understanding and decision consistency. These enhancements would improve MASC's adaptability and robustness in dynamic manufacturing environments. Additionally, the integration of knowledge graphs, digital twins, and edge computing for decentralized scheduling could further extend MASC's applicability to other domains.

CRediT authorship contribution statement

Zelong Wang: Writing – original draft, Validation, Software, Methodology, Conceptualization. **Chenhui Wan:** Writing – original draft, Validation, Software, Methodology, Conceptualization. **Jie Liu:** Supervision, Project administration, Funding acquisition. **Xi Zhang:** Methodology. **Haifeng Wang:** Validation. **Youmin Hu:** Supervision, Project administration, Funding acquisition. **Zhongxu Hu:** Writing – review & editing, Supervision, Project administration, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Table 7
List of abbreviations.

Abbreviation	Definition
FJSP	Flexible Job Shop Scheduling Problem
MASC	Multi-Agent Scheduling Chain
LLMs	Large Language Models
AI	Artificial Intelligence
RL	Reinforcement Learning
HRI	Human-Robot Interaction
NLP	Natural Language Processing
CoT	Chain-of-Thought
LoRA	Low-Rank Adapters
QLoRA	Quantized Low-Rank Adapters
MAS	Multi-Agent Systems
DialBag	Dialogue Bagging
ReAct	Reason + Act
ObsAgent	Observation Agent
SchedAgent	Scheduling Agent
PlanAgen	Planning Agent
CtrlAgent	Control Agent
GA	Genetic Algorithm
MIP	Mixed Integer Programming
FIFO	First In First Out
EET	Earliest End Time
TCP	Tool Center Point

Appendix

A.1. Abbreviations

To enhance readability, a list of all abbreviations used in this paper, along with their definitions, has been compiled, as shown in Table 7. This will help readers quickly understand the specialized terms and acronyms used throughout the manuscript.

A.2. MIP model for FJSP

To address the FJSP within the MASC framework, this study formulates a Mixed Integer Programming (MIP) model [10]. The goal of the MIP model is to minimize the maximum completion time (C_{max}) of all operations, a key objective in job shop scheduling. To ensure the integrity of the scheduling process, the following constraints are imposed:

1. *Operation Assignment Constraint* : This constraint ensures that each operation is assigned to exactly one machine:

$$\sum_{k \in M_{ij}} x_{ijk} = 1, \quad \forall i \in J, \forall j \in O_i \tag{22}$$

where x_{ijk} is a binary decision variable, 1 if operation O_{ij} is processed on machine M_k, otherwise 0, J represents the set of jobs and O_i represents the operations for job i.

2. *Time Arrangement Constraint*: This constraint ensures the continuity of operations, preventing overlap and ensuring that operations are executed sequentially:

$$s_{ijk} \geq s_{i,j-1,k'} + p_{i,j-1,k'} \cdot x_{i,j-1,k'} \tag{23}$$

$$\forall k, k' \in M_{ij}, \forall i \in J, \forall j \in O_i \tag{24}$$

where s_{ijk} is start time of operation O_{ij} on machine M_k, p_{ijk} represents the processing time for operation O_{ij} on machine M_k.

3. *Machine Conflict Constraint*: To prevent multiple operations from being processed on the same machine at the same time, the following constraint is applied:

$$(s_{ijk} + p_{ijk} \cdot x_{ijk} \leq s_{i'j'k}) \vee (s_{i'j'k} + p_{i'j'k} \cdot x_{i'j'k} \leq s_{ijk}) \tag{25}$$

$$\forall i \neq i', j \neq j', \forall k \in M_{ij} \tag{26}$$

This ensures that only one operation is processed on a machine at any given time.

4. *Completion Time Constraint*: The maximum completion time (C_{max}) of all operations is constrained as follows:

$$C_{max} \geq s_{ijk} + p_{ijk} \cdot x_{ijk} \tag{27}$$

$$\forall i \in J, \forall j \in O_i, \forall k \in M_{ij} \tag{28}$$

where C_{max} is the maximum completion time of all operations

The primary objective of the MIP model is to minimize the maximum completion time (C_{max}), which is formulated as:

$$Minimize \quad C_{max} \tag{29}$$

This objective ensures that the entire scheduling process is optimized to minimize the makespan, enhancing the overall efficiency of the system.

The MIP model is solved using the Gurobi solver. While exact algorithms like MIP provide theoretically optimal solutions, they often require significant computational time for large-scale problems. Therefore, the MIP model is included in the algorithm library to provide precise solutions when computational resources and time allow. In cases where faster results are needed, metaheuristic or rule-based algorithms from the MASC tool library can be employed to provide near-optimal solutions efficiently.

Data availability

Data will be made available on request.

References

[1] S. Dauzère-Pérès, J. Ding, L. Shen, K. Tamssaouet, The flexible job shop scheduling problem: A review, European J. Oper. Res. 314 (2) (2024) 409–432.
[2] J. Xie, X. Li, L. Gao, L. Gui, A hybrid genetic tabu search algorithm for distributed flexible job shop scheduling problems, J. Manuf. Syst. 71 (2023) 82–94.
[3] J. Gao, T. Xie, R. Li, K. Wang, L. Zhao, APM: Adaptive parameter multiplexing for class incremental learning, Expert Syst. Appl. 258 (2024) 125135.
[4] W. Ren, Y. Yan, Y. Hu, Y. Guan, Joint optimisation for dynamic flexible job-shop scheduling problem with transportation time and resource constraints, Int. J. Prod. Res. 60 (18) (2022) 5675–5696.
[5] L. Ding, Z. Guan, M. Rauf, L. Yue, Multi-policy deep reinforcement learning for multi-objective multiplicity flexible job shop scheduling, Swarm Evol. Comput. 87 (2024) 101550.
[6] C.L. Liu, C.J. Tseng, P.H. Weng, Dynamic job-shop scheduling via graph attention networks and deep reinforcement learning, IEEE Trans. Ind. Informat. 20 (6) (2024) 8662–8672.
[7] R. Wang, G. Wang, J. Sun, F. Deng, J. Chen, Flexible job shop scheduling via dual attention network-based reinforcement learning, IEEE Trans. Neural Netw. Learn. Syst. 35 (3) (2023) 3091–3102.
[8] H. Oliff, Y. Liu, M. Kumar, M. Williams, M. Ryan, Reinforcement learning for facilitating human–robot-interaction in manufacturing, J. Manuf. Syst. 56 (2020) 326–340.
[9] X. Wang, L. Zhang, T. Lin, C. Zhao, K. Wang, Z. Chen, Solving job scheduling problems in a resource preemption environment with multi-agent reinforcement learning, Robot. Comput.-Integr. Manuf. 77 (2022) 102324.
[10] K. Lei, P. Guo, W. Zhao, Y. Wang, L. Qian, X. Meng, et al., A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem, Expert Syst. Appl. 205 (2022) 117796.
[11] Z. Qin, Y. Lu, Knowledge graph-enhanced multi-agent reinforcement learning for adaptive scheduling in smart manufacturing, J. Intell. Manuf. (2024) 1–24.
[12] M. Panzer, B. Bender, Deep reinforcement learning in production systems: a systematic literature review, Int. J. Prod. Res. 60 (13) (2022) 4316–4341.
[13] J. Achiam, S. Adler, S. Agarwal, et al., GPT-4 technical report, 2023, arXiv: 2303.08774.
[14] C. Chen, K. Zhao, J. Leng, C. Liu, J. Fan, P. Zheng, Integrating large language model and digital twins in the context of industry 5.0: Framework, challenges and opportunities, Robot. Comput.-Integr. Manuf. 94 (2025) 102982.
[15] R. Yang, J. Zhu, J. Man, H. Liu, L. Fang, Y. Zhou, GS-KGC: A generative subgraph-based framework for knowledge graph completion with large language models, Inf. Fusion 117 (2025) 102868.
[16] J.A.H. Ali, B. Gaffinet, H. Panetto, Y. Naudet, Cognitive systems and interoperability in the enterprise: A systematic literature review, Annu. Rev. Control. 57 (2024) 100954.

[17] S. Holt, T. Liu, M.van.der. Schaar, Automatically learning hybrid digital twins of dynamical systems, Adv. Neural Inf. Process. Syst. 37 (2025) 72170–72218.
[18] P. Agrawal, S. Vasania, C. Tan, Can LLMs Perform Structured Graph Reasoning Tasks?, Springer, Cham, 2025, pp. 287–308.
[19] J. Liang, W. Huang, F. Xia, et al., Code as policies: Language model programs for embodied control, in: Proc. 2023 IEEE Int. Conf. Robotics and Automation, ICRA, 2023, pp. 9493–9500.
[20] Y. Jin, D. Li, A. Yong, et al., Robotgpt: Robot manipulation learning from chatgpt, IEEE Robot. Autom. Lett. 9 (3) (2024) 2543–2550.
[21] P. Liu, L. Qian, X. Zhao, B. Tao, Joint knowledge graph and large language model for fault diagnosis and its application in aviation assembly, IEEE Trans. Ind. Informat. 20 (6) (2024) 8160–8169.
[22] H. Zhu, P. Tiwari, A. Ghoneim, M.S. Hossain, A collaborative AI-enabled pretrained language model for aIoT domain question answering, IEEE Trans. Ind. Informat. 18 (5) (2021) 3387–3396.
[23] J. Wei, X. Wang, et al., Chain-of-thought prompting elicits reasoning in large language models, Adv. Neural Inf. Process. Syst. 35 (2022) 24824–24837.
[24] J. Tang, Y. Liu, K.Y. Lin, L. Li, Process bottlenecks identification and its root cause analysis using fusion-based clustering and knowledge graph, Adv. Eng. Inf. 55 (2023) 101862.
[25] T. Wang, J. Fan, P. Zheng, An LLM-based vision and language cobot navigation approach for human-centric smart manufacturing, J. Manuf. Syst. 75 (2024) 299–305.
[26] L. Huang, W. Yu, W. Ma, et al., A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions, 2023, arXiv:2311.05232.
[27] T.B. Brown, Language models are few-shot learners, 2020, arXiv:2005.14165.
[28] E.J. Hu, Y. Shen, P. Wallis, et al., LoRA: Low-rank adaptation of large language models, in: Proc. Int. Conf. Learn. Representations, ICLR, 2022.
[29] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer, Qlora: Efficient finetuning of quantized LLMs, in: Adv. Neural Inf. Process. Syst, NIPS, Vol. 36, 2024.
[30] L. Gan, W. Chu, G. Li, X. Tang, K. Li, Large models for intelligent transportation systems and autonomous vehicles: A survey, Adv. Eng. Inf. 62 (2024) 102786.
[31] B. Ni, M.J. Buehler, MechAgents: Large language model multi-agent collaborations can solve mechanics problems, Gener. New Data, Integr. Knowl. Extrem. Mech. Lett. 67 (2024) 102131.
[32] T. Xiao, P. Xu, Exploring automated energy optimization with unstructured building data: A multi-agent based framework leveraging large language models, Energy Build. 322 (2024) 114691.
[33] B. Liu, C.H. Chen, Z. Wang, A multi-hierarchical aggregation-based graph convolutional network for industrial knowledge graph embedding towards cognitive intelligent manufacturing, J. Manuf. Syst. 76 (2024) 320–332.
[34] X. Zhang, Y. Yang, X. Zhang, Y. Hu, H. Wu, M. Li, H. Handroos, H. Wang, B. Wu, A multi-level digital twin construction method of assembly line based on hybrid worker digital twin models, Adv. Eng. Inf. 62 (2024) 102597.
[35] X. Zhang, X. Zhang, J. Liu, B. Wu, Y. Hu, Graph features dynamic fusion learning driven by multi-head attention for large rotating machinery fault diagnosis with multi-sensor data, Eng. Appl. Artif. Intell. 125 (2023) 106601.
[36] S. Yao, J. Zhao, D. Yu, et al., React: Synergizing reasoning and acting in language models, in: Proc. Int. Conf. Learn. Representations, ICLR, 2023.
[37] N. Ho, L. Schmid, S. Yun, Large language models are reasoning teachers, in: Proc. 61st Annu. Meet. Assoc. Comput. Linguistics, ACL, 2023, pp. 14852–14882.
[38] J. Bai, S. Bai, Y. Chu, et al., Qwen technical report, 2023, arXiv:2309.16609.
[39] A. Zeng, X. Liu, Z. Du, et al., GLM-130b: An open bilingual pre-trained model, in: Proc. Int. Conf. Learn. Representations, ICLR, 2023.
[40] C. Li, et al., ModelScope-agent: Building your customizable agent system with open-source large language models, 2023, arXiv:2309.00986.
[41] Z. Cai, M. Cao, et al., InternLM2 technical report, 2024, arXiv:2403.17297.
[42] P. Brandimarte, Routing and scheduling in a flexible job shop by tabu search, Ann. Oper. Res. 41 (3) (1993) 157–183.
[43] J.W. Barnes, J.B. Chambers, Flexible Job Shop Scheduling By Tabu Search, Graduate Program in Operations and Industrial Engineering, Technical Report Series, ORP, The University of Texas at Austin, 1996, 96-09.
[44] S. Dauzère-Pérès, J. Paulli, An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search, Ann. Oper. Res. 70 (1997) 281–306.
[45] J. Hurink, B. Jurisch, M. Thole, Tabu search for the job-shop scheduling problem with multi-purpose machines, Oper. Res. Spektrum 15 (1994) 205–215.
[46] A.I. Meta, Introducing meta llama 3: The most capable openly available LLM to date, 2024, Meta AI.
[47] P. Clark, I. Cowhey, et al., Think you have solved question answering? try arc, the ai2 reasoning challenge, 2018, arXiv:1803.05457.
[48] R. Zellers, A. Holtzman, et al., HellaSwag: Can a machine really finish your sentence? in: Proc. 57th Annu. Meet. Assoc. Comput. Linguistics, Florence, Italy, 2019, pp. 4791–4800.
[49] D. Hendrycks, C. Burns, et al., Measuring massive multitask language understanding, in: Proc. Int. Conf. Learn. Representations, ICLR, 2021.